SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA1407 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I D K SUMALATHA (192424023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D K SUMALATHA

Annexure A

SCENARIO-BASED TASK

QUESTIONS:

1.Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decision should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.

2.Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression id + id - id, but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decision should be taken to enforce correct associativity and ensure accurate parsing.

3.Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression a + * b from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decision should be taken to improve error detection and recovery while maintaining clarity in parsing.

4. Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar S → if E then S | if E then S else S | a. While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.

5.Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar E → E + E | E * E | id. While parsing the input id + id * id, the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer processes this input, identify the issue caused by the grammar, and explain how precedence and associativity rules should be applied. Based on this, suggest what changes must be made to the grammar or parsing technique to ensure correct evaluation.

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Scenario: Parsing Table Conflict in LL(1)

In an LL(1) parser, the parsing table is constructed using the **FIRST** and **FOLLOW** sets of the grammar. The **FIRST** set determines the terminals that can appear at the beginning of strings derived from a production, while the **FOLLOW** set contains terminals that can appear immediately after a non-terminal. During parsing table construction, each production is placed in the table based on these sets.

A conflict occurs when **more than one production is entered into the same table cell** for a non-terminal and an input symbol. This usually happens because the grammar contains **left recursion**, **common prefixes (left factoring problem)**, or is **ambiguous**. As a result, the parser cannot uniquely determine which production should be applied.

Since LL(1) parsing is deterministic, every table entry must contain **only one production**. Multiple entries make the parser non-deterministic and may lead to incorrect parsing or parser failure.

Decision: The grammar should be modified by **eliminating left recursion**, **performing left factoring**, and ensuring that the FIRST and FOLLOW sets do not overlap. This converts the grammar into a valid **LL(1) grammar** with a conflict-free parsing table.

---

2. Scenario: Incorrect Parse Tree Generation

The syntax analyzer constructs a parse tree by applying grammar productions according to the input tokens. For the expression **id + id - id**, the parse tree determines the order in which operations are evaluated.

If the grammar does not specify operator associativity, the expression may be grouped incorrectly. For example, it may be interpreted as **id + (id - id)** instead of the correct **(id + id) - id**. This results in incorrect evaluation because the operators are not associated properly.

Grammar rules directly influence the structure of the parse tree. A grammar that does not enforce associativity allows multiple valid parse trees for the same expression.

Decision: The grammar should enforce **left associativity** for addition and subtraction. For example:

- $E \rightarrow E + T \mid E - T \mid T$
- $T \rightarrow id$

This ensures expressions are evaluated from **left to right**, producing the correct parse tree and accurate results.

---

3. Scenario: Invalid Expression Handling

The lexical analyzer correctly identifies the tokens in the expression **a + * b** as identifiers and operators. However, the syntax analyzer checks whether these tokens follow the grammar rules.

The error occurs because the grammar expects an **operand after the '+' operator**, but another operator '*' appears instead. Therefore, the token sequence violates the grammar, making it impossible to construct a valid parse tree.

In **predictive parsing (LL)**, the parser detects the error when no valid production exists in the parsing table for the current input symbol. In **shift-reduce parsing (LR)**, the parser detects the error when no valid shift or reduce action is available.

Without proper recovery, the parser stops after reporting the error.

Decision: The parser should implement **error detection and recovery techniques**, such as **panic-mode recovery**, **phrase-level recovery**, or **synchronization using FOLLOW sets**. These methods allow the parser to continue analyzing the remaining input while providing meaningful error messages.

---

4. Scenario: Ambiguity in Conditional Statements

The grammar

- S → if E then S
- S → if E then S else S
- S → a

is ambiguous because the parser cannot determine which **if** statement an **else** belongs to. This is known as the **dangling else problem**.

When constructing the parsing table, conflicts arise because both productions begin with the same prefix:

if E then S

The parser cannot decide whether to reduce using the first production or continue parsing the second production containing **else**.

This ambiguity affects deterministic parsing by introducing parsing conflicts and producing multiple possible parse trees.

Decision: The grammar should be rewritten to distinguish **matched** and **unmatched** statements, for example:

- S → M | U
- M → if E then M else M | a
- U → if E then S | if E then M else U

Alternatively, parser generators often resolve the ambiguity by associating **else with the nearest unmatched if**, which is the standard rule used in most programming languages.

---

5. Scenario: Operator Precedence Conflict

The grammar

- E → E + E
- E → E * E
- E → id

is ambiguous because it does not define **operator precedence** or **associativity**.

For the input **id + id * id**, two different parse trees are possible:

- (id + id) * id
- id + (id * id)

The correct evaluation requires multiplication to have higher precedence than addition. Since the grammar does not specify this rule, the parser cannot determine the correct parse tree.

Operator precedence and associativity are essential to ensure expressions are evaluated correctly.

Decision: The grammar should be rewritten to enforce precedence, for example:

- E → E + T | T
- T → T * F | F
- F → id

Here:

- **Multiplication has higher precedence** than addition.
- **Addition and multiplication are left associative.**

Alternatively, parser generators can use **precedence and associativity declarations** to resolve such conflicts automatically.

8/8/

Assessment - 3

Scenario Based Task

Name: D.K. Sumalatha
Reg.NO: 192424023
Date: 7/8/26
Course code: CSA1402
Complier design.

1) Scenario: Parsing Table conflict in LL(1)

In an LL(1) parser syntax analyser construct parsing table using the FIRST and FOLLOW sets of grammar. The FIRST set determines terminals that can appear at the beginning strings derived from a production while Follow set contains terminals that can appear immediately after non-terminal During parsing table construction each production placed table placed sets.

First (α) symbols $\alpha \Rightarrow \varepsilon$ then Follow (A)

$$A \rightarrow \alpha / \beta$$

$$FIRST(\alpha) \cap FIRST(\beta) \neq \phi$$

A conflict occurs when more than one production entered into parsetable cell non-terminal input symbol. This usually happens because grammar contains left recursion common prefix (Left factoring problem) ambiguous cannot uniquely production should applied since LL(1) parsing deterministic every table entry must contain only one production multiple entries parser non-determine incorrect parsing parser failure.

Multiple entires LL(1) parsing table cell indicate the grammar not LL(1). The grammar should modified using left factoring left recursion eliminated note table

cell one production.

2) Scenario:

Incorrect parse Tree Generation.

The syntax analyzer construct parse tree by applying grammar productions according to the input tokens. For the expression id+id-id, the parse tree determines order operations are evaluated.

If grammar does not specify operator associativity expression grouped incorrectly $E \rightarrow E+E \mid E-E \mid id$.

For input id+id-id grammar generate two different interpretation (id+id)-id

id+(id-id) Grammar ambiguous not specify + and - grouped

The operators + & - normally follow left to right (id+id)-id Decision. The grammar should enforce left associativity for addition & subtraction $E \rightarrow E+T \mid E-T \mid T$

$T \rightarrow id$

This ensure expression evaluate left to right producing correct parse tree accurate result.

---

3) Scenario:

Invalid Expression Handling:-

The lexical analyzer correctly identifies tokens expression a+*b identifiers and operators, syntax analyser check whether tokens follow grammar rules.

The error occurs because the grammar expects operand after the + operator but another operator appears * instead. The token sequence violates grammar impossible to construct valid parse tree.

Scanned with OKEN Scanner

In predictive parsing (LL) the parser detects error when no valid production exists parsing table current input symbol. In shift reduce parsing (LR) parser detects error when no valid shift or reduce action available. Without proper recovery parser stops after reporting error.

Decision: The parser should implement error detection recovery techniques panic mode recovery phrase level recovery or synchronization Follow sets the methods allow parser continue analyzing remaining input providing meaningful error messages.

---

4) scenario:
Ambiguity in conditional statements:

The grammar $S \rightarrow$ if E then S
$S \rightarrow$ if E then S else S
$S \rightarrow a$ is ambiguous because, parser cannot determine if statement else belongs. This known dangling else problem. When constructing parsing table conflicts arise because both production begin same prefix.

If E then S: The parser cannot decide whether reduce using 1st production or continue parsing the second production containing else.

This ambiguity affects deterministic parsing by introducing parsing conflicts producing multiple possible parse trees.

Decision:- The grammar should be rewritten distinguish matched and unmatched statements for example, $S \rightarrow M/U$
$M \rightarrow$ if E then M else M/a

Scanned with OKEN Scanner

U → If E then S | if E then M else U

Alternatively parser generators often resolve the ambiguity associating else nearest unmatched if which is the standard rule used in most programming language.

5) Scenario: Operator precedence conflict.

The grammar: E → E+E, E → E * E, E → id

It is ambiguous because do not define operator precedence or associativity.

For input id+id*id two different parse tree are possible.

* (id + id) * id
* id + (id * id)

The correct evaluation requires multiplication have higher precedence than addition. since the grammar does not specify rule, parser cannot determine correct parse tree. Operator precedence associativity essential ensure expression evaluated correctly.

Decision: The grammar should be rewritten enforce precedence for example:

E → E+T / T
T → T * F / F
F → id

* Multiplication has higher precedence than addition
* Addition & multiplication left associative.

Alternatively parser generators use precedence associativity declaration resolve conflicts automatically.

Scanned with OKEN Scanner